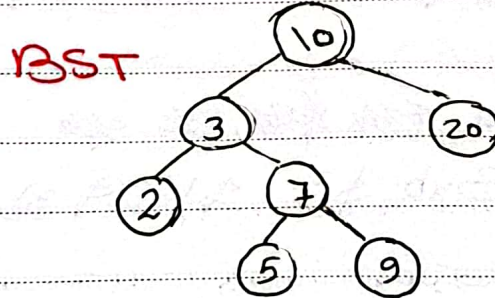


محاضرة الخوارزميات 2 نظري

Binary Search tree (BST)

هي شجرة ثنائية حققت شروط معينة لحق مبادئ
BST وهذه الشروط (القيود) على Data
الشجرة الثنائية العادية، القيود كانت على الشكل
قيود أو شروط، الـ BST هي
ان يكون لكل عقدة

← كل شيء على يسار العقدة، القيم يعني اصغر منها
الـ Subtree اليسار كله اصغر
← كل شيء على يمين العقدة، القيم يعني أكبر منها
الـ Subtree اليمين كله أكبر



الفائدة:

الـ tree الثنائية للبحث Search عادي لازم امشي على كل
العقد فيكون لقيته n

أما الـ BST الـ Search يكون اسرع لاننا مو بشرط امشي
 n مرة لاننا يكون عم احسن أحد، الاطراف بكل مرة

طريقة التعريف (code)

طريقة أولى : شكل عودي

هي أصغر بلقي بيا

node

boolean

Node Search(Node node, ^{مثلا} int x) {

if (node == null)

return null;

if (node.Value == x)

return node;

if (node.Value > x)

return Search(node.left, x);

~~return node~~ else

return Search(node.right, x);

}

لازم اعل return بكل فرع من أفرع التابع العودي لاني كل

return هي مستوى فالا لازم كل مستوى يرجع شي

لازم كل الأفرع ترجع قيمة ← null

node ←

أو ممكن خزن القيمة بـ فتحول ورجعو آخري شي لازم استخدام منو

المعودية لازم جاوب على 3 أسئلة

← سؤال الأول: (القيمة الـ node أولا) في المثال

← سؤال جدي: حلو كل المسألة كافي (البحث في اليسار) في المثال

← ركب حل من السؤال الجدي (البحث في اليمين) في المثال

2. طريقة تايبة : غير عودية

```
Node Search(Node node, int x) {  
    مثال  
    while (node != null && node.value != x)  
    {  
        if (node.value > x)  
            node = node.left;  
        else  
            node = node.right;  
    }  
    return node;  
}
```

راجع إما node أو null لان node = null إذا افشل الشرط

التحقيق :

احسن الاحوال $O(1)$ لصيغ $root$ وخلص
الاسود $O(h)$ لاننا نطعم احذف بعلامة دفعه تماماً
لاننا tree خطية يمكن انما اذا كانت Full tree يكون
العقيد $\log_2(n)$ لانوع احذف دفعه احباري
فيه

$O(h)$ يعني من رتبة الارتفاع تبع tree بناذا كان
لا ارتفاع n يعني خطية يكون العقيد
من رتبة $O(n)$ فالتحقيق بعيد على ارتفاع
الشجرة

أي واحد أحسن عودي أو غير عودي ؟؟

الحل غير العودي هو الأفضل إذا كانت الطريقتين نفس بعض

بالزمن مثلاً ما تواجه مشاكل ال Stack + الغير عودي

أسرع لأن بلا Stack يعني عمل push و pop

+ الحل العودي ينفذ ولها كانت عدد العقد و

ما تواجه مشكلة StackOverflow

- الإضافة في BST

← بالأوراق ورقة وصل على مكان null ضيف

← كعقدة داخلية يضاف بمكان المناسب حسب tree

إذا بقي صنفه قيمة موجبة سابقاً إما ينفذ كعقدة يكون

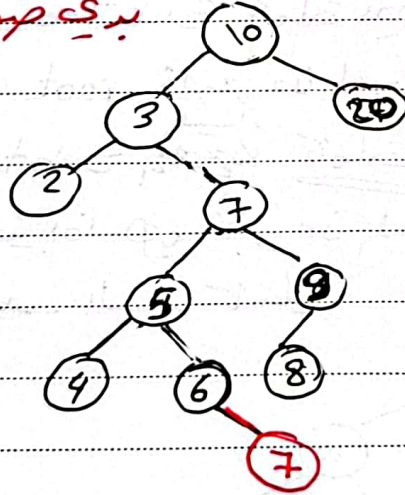
حالة عدد لم مرة تكررت لفئة أو لحل الاستدع عرفها

للإضافة تسع بالسكر فيكون

عين العقدة أكبر يسار العقدة أصغر أو يساوي

المساواة بأحد الطرفين

بدي صنف (7)



تضاف في مكان

الصحيح بـ

tree

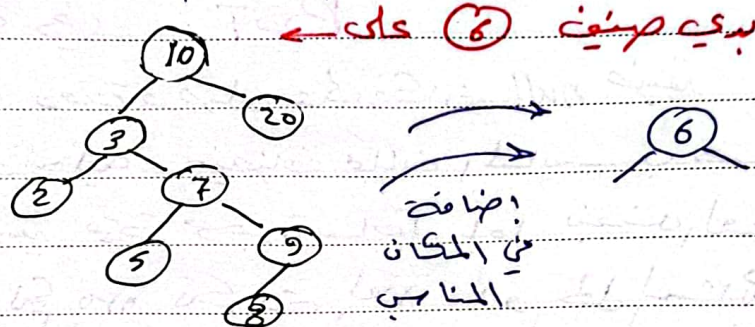
ليس خطية المسواة في أحد الطرفين !!

مستند وقت البحث موزن على مكان واحد لاننا لننضمه والإضافة

يمكننا واحد واحد لا طرف يعني

→ إضافة root

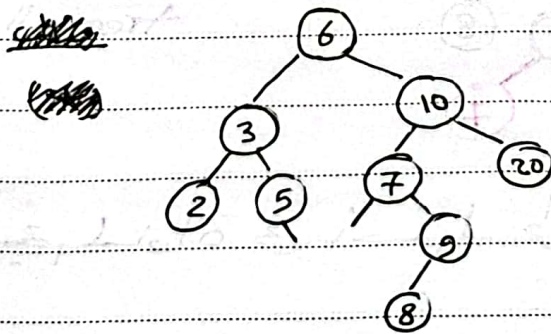
طافني حطو بدك root وخليها تأشير عليه
وبعد لاننا لازم حقت الشرح بتبع BST كله ليسا، أصغر
كل، لهين أكبر يعني لازم نفرط tree وأرجع ركب



عشي عقدة عقدة وضيف لكن، كل بيتر وهو الأمثل
لاننا المقيد راج يكون الأمثل حالة $n \log(n)$ والاسود $n \cdot n$

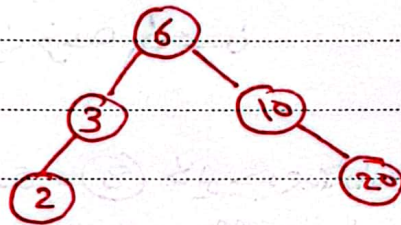
الطريقة الأمثل للإضافة

هي إضافة Subtree كاملة يعني أوشي بقارت مع root
إذا كان root أصغر من العدد ملزود إضافة باخذ root
مع root Subtree كاملة بدون التحقق من الفرع
وضيفه لاننا إذا هي أكبر من root فهي حقاً أكبر
من أقيم ~~اليسار~~ تماماً

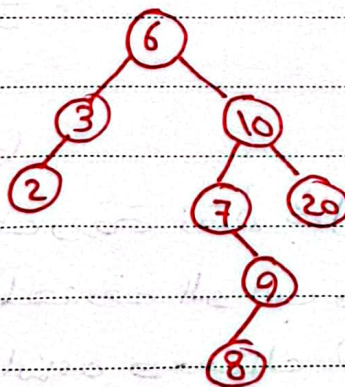


شرح توضيحي :

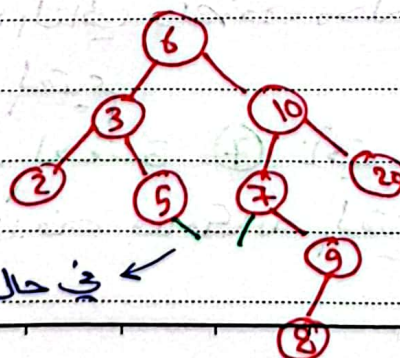
عند إضافة عقدة ما لشجرة البحث الثنائي كجذر يجب المحافظة على خصائص الـ BST وهي أن جميع القيم على اليمين أكبر وجميع القيم على اليسار أصغر. نقارن القيمة الجديدة مع الجذر القديم. إذا كانت القيمة أصغر نأخذ الفرع الأيسر مع الجذر الجديد. كأمثلة (يعني المثال بأخذ عشرة وكلمتي أكبر منها) ويصبح فرع عيني للقيمة الجديدة بمقارنته الفرع الآخر (بالمثال 3 نأخذها مع كلمتي أصغر منها)



بعدها نرجع بمقارنته 7 هي أكبر أخذها هي وكلمتي أكبر منها



نرجع بمقارنته 5 هي أصغر بأخذها هي وكلمتي أصغر منها



← في حال وجود قيم إضافية

ملاحظات

* أسلوب الإضافة في كعقدة داخلية يزيد h فهو غير مناسب لأن الهدف أن h تكون أقل ما يمكن ليكون العقيد أفضل ما يمكن عند إضافة عقدة ما ويمكن تجنب عنها أو الحاجة إيجادها بهذا البرنامج يفضل إضافتها كجذر إذا لا نضيف كعقدة أسرع سؤال ما هو أول عنصر أكبر من عقدة ما (على خوار x)

هو أصغر عنصر في الفرع الأيمن

مثاله ما هو العنصر التالي للعقدة ⑦

هو أصغر عنصر في فرعه الأيمن

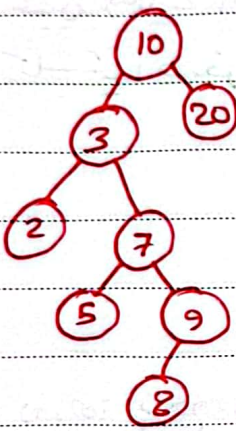
يعني ⑧

يلب ما هو التالي للعقدة ⑧ لا يوجد subtree

أيضا ف نرجع بالآباء أول أب أكبر هو العنصر

التالي إذا لم نجد بالآباء قتل ②٥ فهي

أكبر عنصر بالشجرة



مثال ⑧

الحذف في BST

حالة ① أبسط حالة هي حذف ورقة نقطه نجعل الاب يأشر على $Null$

حالة ② إذا كان المحذوف هو ابن واحد

نجعل الاب يأشر على ابائه - مثال ②

حالة ③ إذا كان المحذوف له اثنين

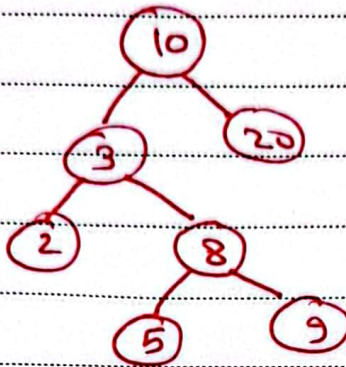
④ نأيت بالعنصر التالي ونضعه محله

ونحذفه من مكانه القديم

مثال نريد حذف لعقدة ⑦ نأيت بالعنصر التالي وهو ⑧

نضعه محله ونحذفه من مكانه القديم (نحل $replace$)

رصف الشجرة بعد الحذف



لغقيد الحذف حسب الحالة

حالة [1] $O(1)$ leave

حالة [2] $O(1)$ عقدة لا ابن

حالة [3] $O(h)$ وهو لغقيد الجداد ~~في~~ الغنص التالي

+ $O(1)$ لعملية الاستبدال

+ $O(1)$ الحذف

الـ h ممكن تكون n اذ $\log(n)$ او - - -

لغقيد الغنص على الارتفاع التحدي في الاشجار البحث

الشابية هو ~~ممكن~~ ما يكون الـ h كبيرة

ف مقي افرض ~~هتود~~ افرضية طبق اضبط الـ h

ما يطول مثل ان افرضه فيد ان تكون متوازنة

لانو اذا صار الـ h من مرتبة n رجعت tree

خضية ما استغنت شي نـ